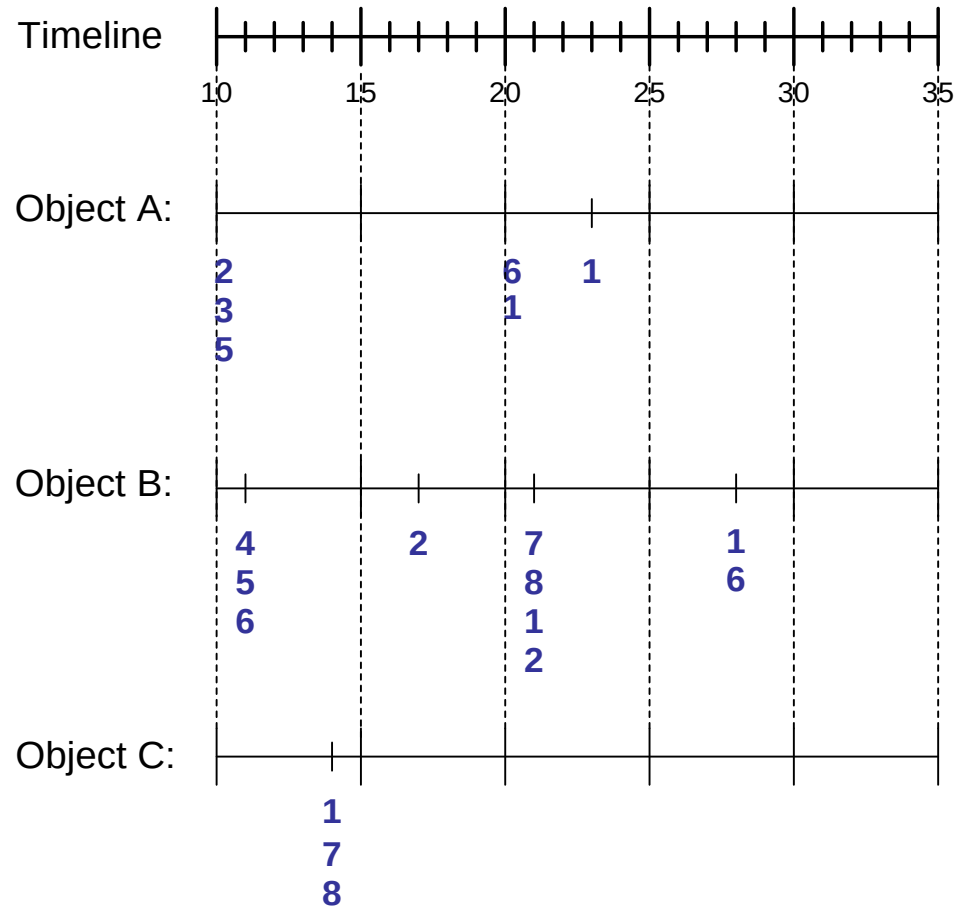


Sequential Pattern Mining

Sequence Data

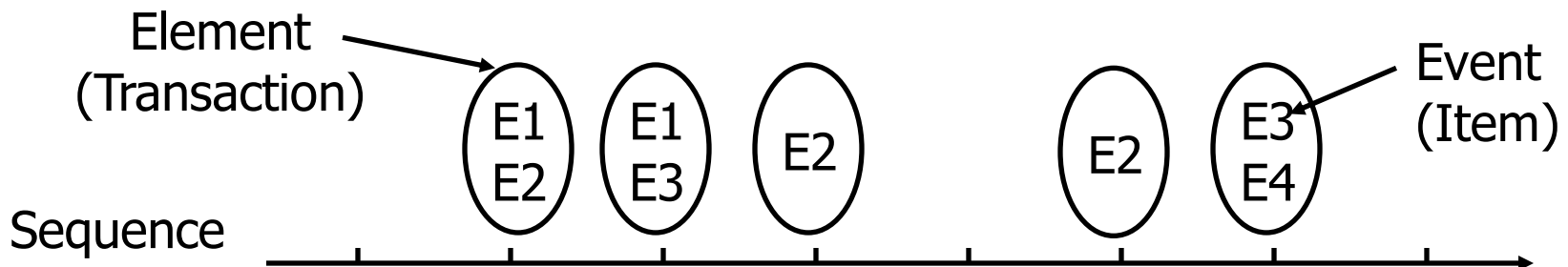
Sequence Database:

Object	Timestamp	Events
A	10	2, 3, 5
A	20	6, 1
A	23	1
B	11	4, 5, 6
B	17	2
B	21	7, 8, 1, 2
B	28	1, 6
C	14	1, 8, 7



Examples of Sequence Data

Sequence Database	Sequence	Element (Transaction)	Event (Item)
Customer	Purchase history of a given customer	A set of items bought by a customer at time t	Books, diary products, CDs, etc
Web Data	Browsing activity of a particular Web visitor	A collection of files viewed by a Web visitor after a single mouse click	Home page, index page, contact info, etc
Event data	History of events generated by a given sensor	Events triggered by a sensor at time t	Types of alarms generated by sensors
Genome sequences	DNA sequence of a particular species	An element of the DNA sequence	Bases A,T,G,C



Formal Definition of a Sequence

- A sequence is an ordered list of elements (transactions)

$$s = \langle e_1 e_2 e_3 \dots \rangle$$

- Each element contains a collection of events (items)

$$e_i = \{i_1, i_2, \dots, i_k\}$$

- Each element is attributed to a specific time or location
- Length of a sequence, $|s|$, is given by the number of elements of the sequence
- A k-sequence is a sequence that contains k events (items)

Formal Definition of a Subsequence

- A sequence $\langle a_1 a_2 \dots a_n \rangle$ is contained in another sequence $\langle b_1 b_2 \dots b_m \rangle$ ($m \geq n$) if there exist integers $i_1 < i_2 < \dots < i_n$ such that $a_1 \subseteq b_{i_1}$, $a_2 \subseteq b_{i_2}$, ..., $a_n \subseteq b_{i_n}$

Data sequence	Subsequence	Contain?
$\langle \{2,4\} \{3,5,6\} \{8\} \rangle$	$\langle \{2\} \{3,5\} \rangle$	Yes
$\langle \{1,2\} \{3,4\} \rangle$	$\langle \{1\} \{2\} \rangle$	No
$\langle \{2,4\} \{2,4\} \{2,5\} \rangle$	$\langle \{2\} \{4\} \rangle$	Yes

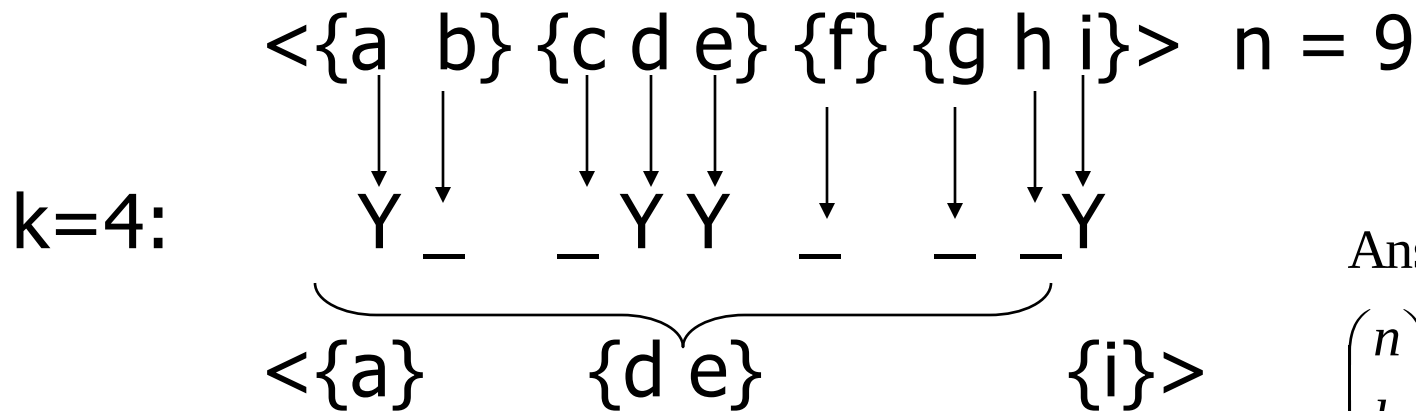
- The support of a subsequence w is defined as the fraction of data sequences that contain w
- A *sequential pattern* is a frequent subsequence (i.e., a subsequence whose support is $\geq \text{minsup}$)

Sequential Pattern Mining: Definition

- Given:
 - a database of sequences
 - a user-specified minimum support threshold, *minsup*
- Task:
 - Find all subsequences with support $\geq \textit{minsup}$

Sequential Pattern Mining: Challenge

- Given a sequence: $\langle \{a\ b\} \{c\ d\ e\} \{f\} \{g\ h\ i\} \rangle$
 - Examples of subsequences:
 $\langle \{a\} \{c\ d\} \{f\} \{g\} \rangle$, $\langle \{c\ d\ e\} \rangle$, $\langle \{b\} \{g\} \rangle$, etc.
- How many k-subsequences can be extracted from a given n-sequence?



Answer :

$$\binom{n}{k} = \binom{9}{4} = 126$$

Challenges on Sequential Pattern Mining

- A **huge** number of possible sequential patterns are hidden in databases
- A mining algorithm should
 - find the **complete set of patterns**, when possible, satisfying the minimum support (frequency) threshold
 - be highly **efficient, scalable**, involving only a small number of database scans
 - be able to incorporate various kinds of **user-specific constraints**

Sequential Pattern Mining Algorithms

- Concept introduction and an initial Apriori-like algorithm
 - Agrawal & Srikant. Mining sequential patterns, ICDE'95
- Apriori-based method: **GSP** (Generalized Sequential Patterns: Srikant & Agrawal @ EDBT'96)
- Pattern-growth methods: FreeSpan & **PrefixSpan** (Han et al.@KDD'00; Pei, et al.@ICDE'01)
- Vertical format-based mining: **SPADE** (Zaki@Machine Learning'00)
- Constraint-based sequential pattern mining (SPIRIT: Garofalakis, Rastogi, Shim@VLDB'99; Pei, Han, Wang @ CIKM'02)
- Mining closed sequential patterns: **CloSpan** (Yan, Han & Afshar @SDM'03)

Extracting Sequential Patterns

- Given n events: $i_1, i_2, i_3, \dots, i_n$
- Candidate 1-subsequences:
 $\langle \{i_1\} \rangle, \langle \{i_2\} \rangle, \langle \{i_3\} \rangle, \dots, \langle \{i_n\} \rangle$
- Candidate 2-subsequences:
 $\langle \{i_1, i_2\} \rangle, \langle \{i_1, i_3\} \rangle, \dots, \langle \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_2\} \rangle, \dots,$
 $\langle \{i_{n-1}\} \{i_n\} \rangle$
- Candidate 3-subsequences:
 $\langle \{i_1, i_2, i_3\} \rangle, \langle \{i_1, i_2, i_4\} \rangle, \dots, \langle \{i_1, i_2\} \{i_1\} \rangle, \langle \{i_1, i_2\} \{i_2\} \rangle, \dots,$
 $\langle \{i_1\} \{i_1, i_2\} \rangle, \langle \{i_1\} \{i_1, i_3\} \rangle, \dots, \langle \{i_1\} \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_1\} \{i_2\} \rangle, \dots$

Generalized Sequential Pattern (GSP)

- **Step 1:**

- Make the first pass over the sequence database D to yield all the 1-element frequent sequences

- **Step 2:**

Repeat until no new frequent sequences are found

- **Candidate Generation:**

- Merge pairs of frequent subsequences found in the $(k-1)th$ pass to generate candidate sequences that contain k items

- **Candidate Pruning:**

- Prune candidate k -sequences that contain infrequent $(k-1)$ -subsequences

- **Support Counting:**

- Make a new pass over the sequence database D to find the support for these candidate sequences

- **Candidate Elimination:**

- Eliminate candidate k -sequences whose actual support is less than *minsup*

Candidate Generation

- Base case ($k=2$):

- Merging two frequent 1-sequences $\langle \{i_1\} \rangle$ and $\langle \{i_2\} \rangle$ will produce two candidate 2-sequences: $\langle \{i_1\} \{i_2\} \rangle$ and $\langle \{i_1 i_2\} \rangle$

- General case ($k>2$):

- A frequent $(k-1)$ -sequence w_1 is merged with another frequent $(k-1)$ -sequence w_2 to produce a candidate k -sequence if the subsequence obtained by removing the first event in w_1 is the same as the subsequence obtained by removing the last event in w_2
 - The resulting candidate after merging is given by the sequence w_1 extended with the last event of w_2 .
 - If the last two events in w_2 belong to the same element, then the last event in w_2 becomes part of the last element in w_1
 - Otherwise, the last event in w_2 becomes a separate element appended to the end of w_1

Candidate Generation Examples

- Merging the sequences
 $w_1 = \langle \{1\} \{2\ 3\} \{4\} \rangle$ and $w_2 = \langle \{2\ 3\} \{4\ 5\} \rangle$
will produce the candidate sequence $\langle \{1\} \{2\ 3\} \{4\ 5\} \rangle$ because the last two events in w_2 (4 and 5) belong to the same element
- Merging the sequences
 $w_1 = \langle \{1\} \{2\ 3\} \{4\} \rangle$ and $w_2 = \langle \{2\ 3\} \{4\} \{5\} \rangle$
will produce the candidate sequence $\langle \{1\} \{2\ 3\} \{4\} \{5\} \rangle$ because the last two events in w_2 (4 and 5) do not belong to the same element
- We do not have to merge the sequences
 $w_1 = \langle \{1\} \{2\ 6\} \{4\} \rangle$ and $w_2 = \langle \{1\} \{2\} \{4\ 5\} \rangle$
to produce the candidate $\langle \{1\} \{2\ 6\} \{4\ 5\} \rangle$ because if the latter is a viable candidate, then it can be obtained by merging w_1 with $\langle \{1\} \{2\ 6\} \{5\} \rangle$

GSP Example

Frequent 3-sequences

< {1} {2} {3} >
< {1} {2 5} >
< {1} {5} {3} >
< {2} {3} {4} >
< {2 5} {3} >
< {3} {4} {5} >
< {5} {3 4} >

Candidate Generation

< {1} {2} {3} {4} >
< {1} {2 5} {3} >
< {1} {5} {3 4} >
< {2} {3} {4} {5} >
< {2 5} {3 4} >

Candidate Pruning

< {1} {2 5} {3} >

Finding Length-1 Sequential Patterns

- Examine GSP using an example
- Initial candidates: all singleton sequences
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
- Scan database once, count support for candidates

$min_sup = 2$

Seq. ID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

Cand	Sup
$\langle a \rangle$	3
$\langle b \rangle$	5
$\langle c \rangle$	4
$\langle d \rangle$	3
$\langle e \rangle$	3
$\langle f \rangle$	2
$\langle g \rangle$	1
$\langle h \rangle$	1

GSP: Generating Length-2 Candidates

51 length-2
Candidates

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Without Apriori
property,
 $8*8 + 8*7/2 = 92$
candidates

Apriori prunes
44.57% candidates

The GSP Mining Process

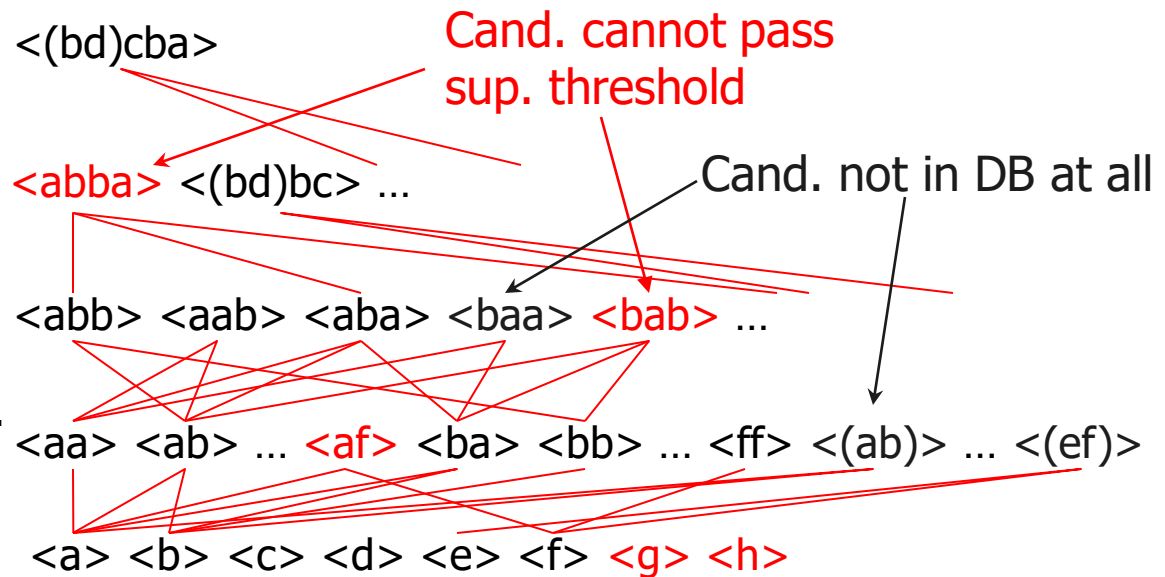
5th scan: 1 cand. 1 length-5 seq.
pat.

4th scan: 8 cand. 6 length-4 seq.
pat.

3rd scan: 47 cand. 19 length-3 seq.
pat. 20 cand. not in DB at all

2nd scan: 51 cand. 19 length-2 seq.
pat. 10 cand. not in DB at all

1st scan: 8 cand. 6 length-1 seq.
pat.



$min_sup = 2$

Seq. ID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Candidate Generate-and-test: Drawbacks

- A huge set of candidate sequences generated.
 - Especially 2-item candidate sequence.
- Multiple Scans of database needed.
 - The length of each candidate grows by one at each database scan.
- Inefficient for mining long sequential patterns.
 - A long pattern grow up from short patterns
 - The number of short patterns is exponential to the length of mined patterns.